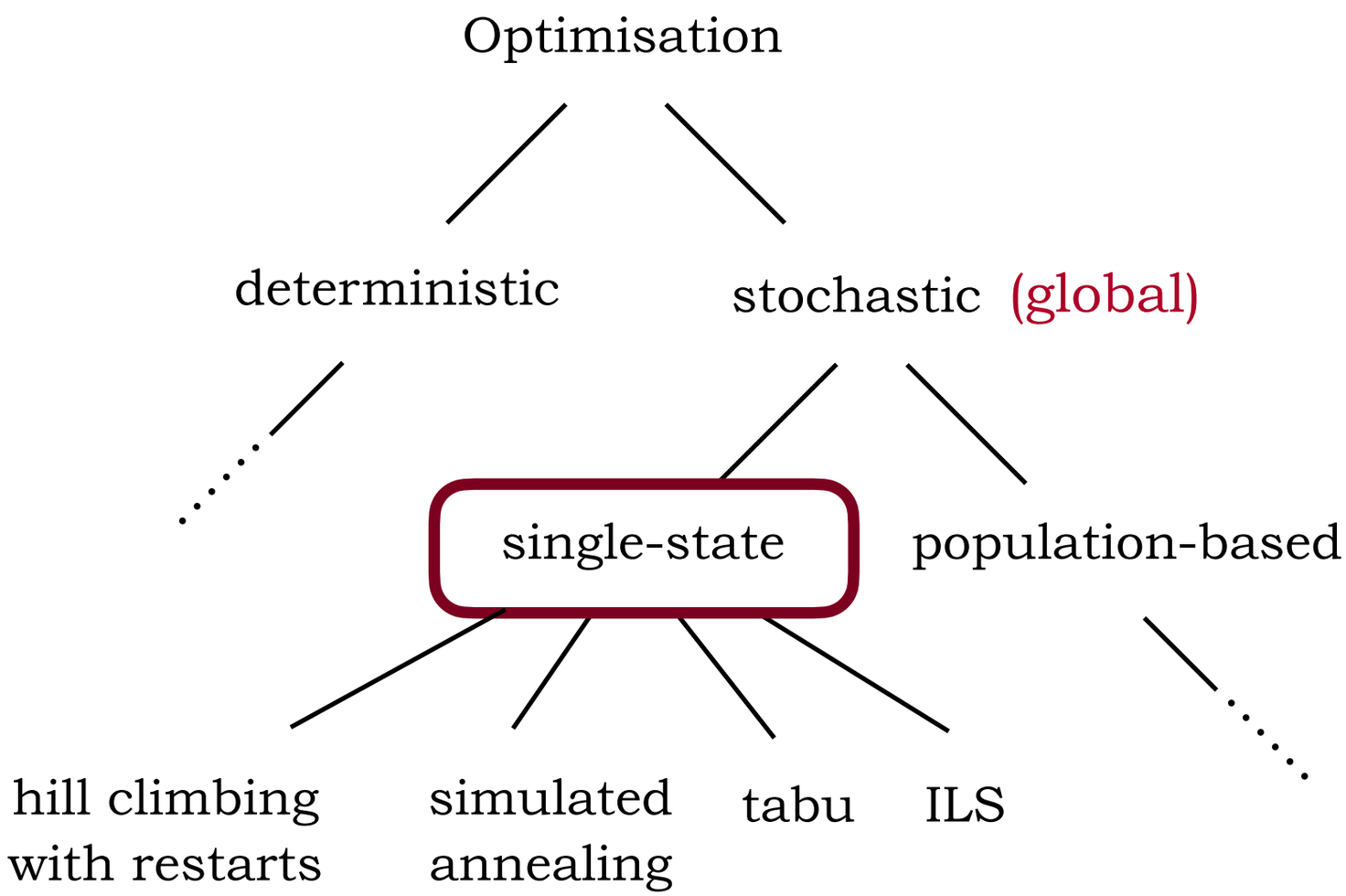
Single-State Global Optimisation

Reference

Essentials of Metaheuristics, 2nd Ed.
Sean Luke, Lulu, 2016

© Siwen Luo, UWA

Optimisation Algorithms - Roadmap



Exploration vs Exploitation

Goal

Combine:

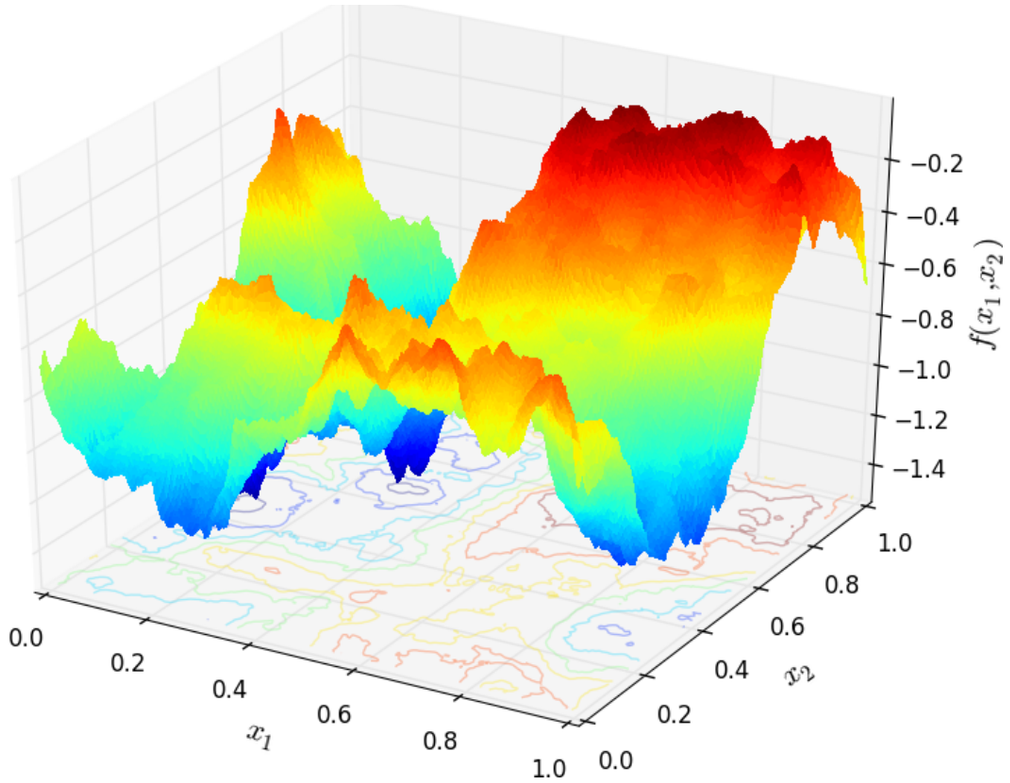
- exploitation - “local” search, “small” tweaks
- exploration - “global” search, any possible change

Recall: Examples of Increasing Exploration

- Adjust modification procedure
- Adjust selection procedure
- “Hyperjumps”/restarts

Search spaces can be rocky!

What is the *best* thing to do?



Taking Stock

- There are many algorithms (with many great names!)
- But in essence they're all balancing local and global search
- Trying to get the most "bang for the buck"
- From the *No Free Lunch Theorem* we know there's no uniformly "best" algorithm
- But different algs try and exploit information they can glean in different ("intelligent") ways
  - these are sometimes called ***meta-heuristics***
  - may work better on some problem classes than others
  - more on this later
- Let's start with one of the simplest combinations…

Hill-Climbing with Restarts

Algorithm 10 *Hill-Climbing with Random Restarts*

```
T ← distribution of possible time intervals

S ← some initial random candidate solution
Best ← S
repeat
    time ← random time in the near future, chosen from T
    repeat
        R ← Tweak(Copy(S))
        if Quality(R) > Quality(S) then
            S ← R
    until S is the ideal solution, or time is up, or we have run out of total time
    if Quality(S) > Quality(Best) then
        Best ← S
    S ← some random candidate solution
until Best is the ideal solution or we have run out of total time
return Best
```

- Time intervals are "short" - weighted towards *exploration*
- Time intervals are "long" - weighted to *exploitation*
- Note scope for eg. starting short and increasing over time

Non-Uniform Tweaks

- Recall b*ounded uniform convolution* (Alg 8) used a linear prob. distribution with bounds - no chance of bigger jumps.

Algorithm 8 *Bounded Uniform Convolution*

```
v⃗ ← vector ⟨v_1, v_2, ... v_l⟩ to be convolved
p ← probability of adding noise to an element in the vector        ▷ Often p = 1
r ← half-range of uniform noise
min ← minimum desired vector element value
max ← maximum desired vector element value

for i from 1 to l do
    if p ≥ random number chosen uniformly from 0.0 to 1.0 then
        repeat
            n ← random number chosen uniformly from −r to r inclusive
        until min ≤ v_i + n ≤ max
        v_i ← v_i + n
return v⃗
```

- can't do bigger jumps
- how do you choose min and max?
- uniform dist. abandons our "locality" heuristic (whether that's good or bad)

Gaussian Tweaks

- Common to do many small tweaks and fewer large tweaks
  - eg. many hill climbs, occasional jumps to look for bigger hills
- Common distribution
  - jumps increasing to any (infinite) size
  - with decreasing probability
  - *Gaussian (Normal) distribution*

$N(\mu, \sigma^2)$

μ - mean
σ - standard deviation

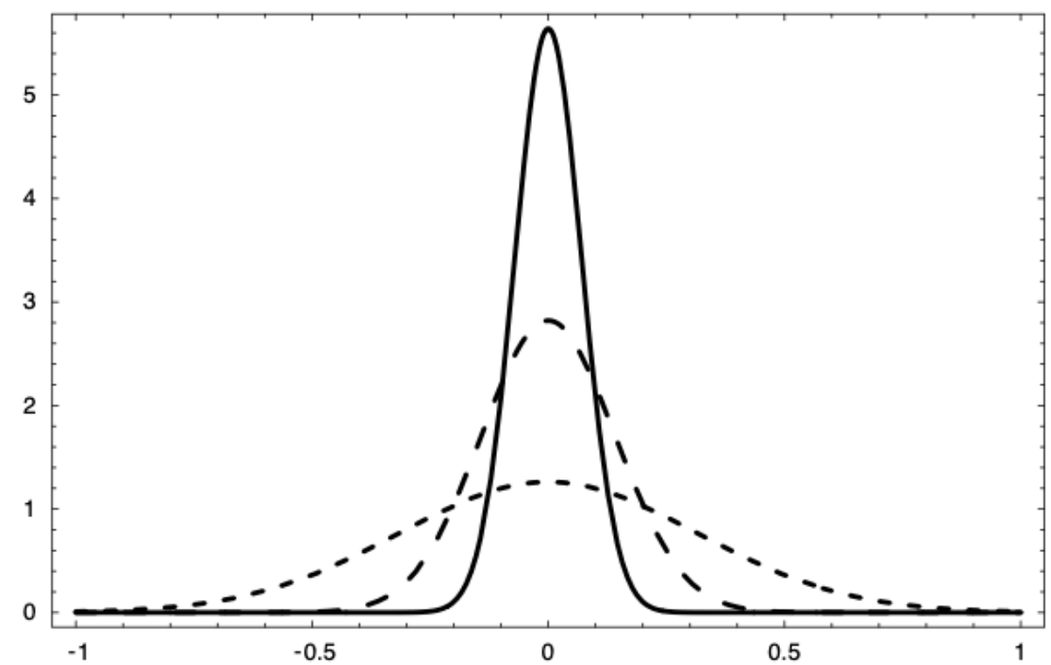


Figure 7 Three Normal or Gaussian distributions $N(\mu, \sigma^2)$ with the mean $\mu = 0$ and the variance σ^2 set to $\sigma^2 = 0.005$: ——, $\sigma^2 = 0.02$: – – –, and $\sigma^2 = 0.1$: - - - -.

Gaussian Tweaks

Algorithm 11 *Gaussian Convolution*

$\vec{v} \leftarrow$ vector $\langle v_1, v_2, ...v_l \rangle$ to be convolved
$p \leftarrow$ probability of adding noise to an element in the vector ▷ Often $p = 1$
$\sigma^2 \leftarrow$ variance of Normal distribution to convolve with ▷ Normal = Gaussian
$min \leftarrow$ minimum desired vector element value
$max \leftarrow$ maximum desired vector element value

may be (-∞,∞)

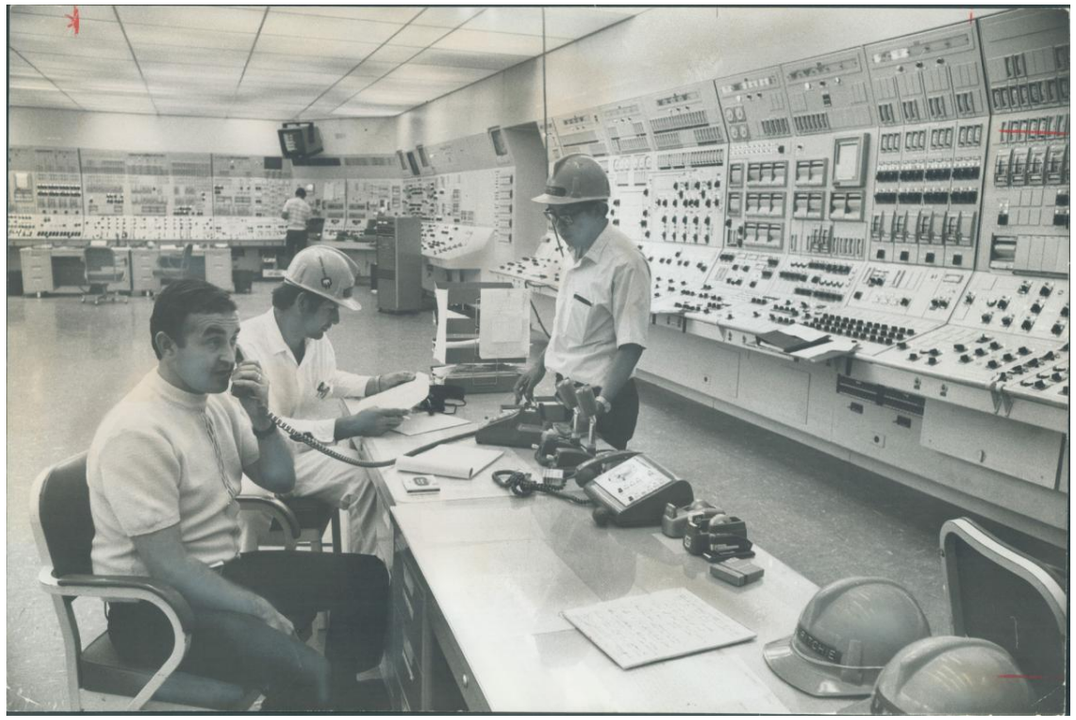
for i from 1 to l **do**
    if $p \geq$ random number chosen uniformly from 0.0 to 1.0 **then**
        repeat
            $n \leftarrow$ random number chosen from the Normal distribution $N(0, \sigma^2)$
        until $min \leq v_i + n \leq max$
        $v_i \leftarrow v_i + n$
return $\vec{v}$

σ (or σ^2) is a (very direct) tuning parameter (hyper-parameter)

- rate of exploration

Interdependence of Hyper-Parameters

- Note: seen a few examples of hyper-parameters that do different things
  - Can't assume they are independent!
  - Interaction may be complex



- Juggling them can be quite an art

Its Subtle!

- σ large → more extreme choices (more exploration)
- n large → more culling of extreme choices (*selective pressure*, less exploration)

Not a direct relationship though

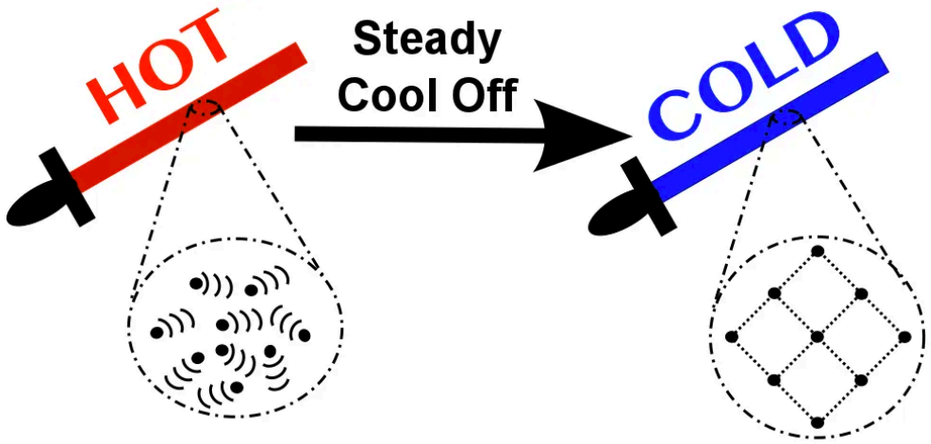
- depends whether candidates from large σ jump to a better peak(s) but not better quality (culled) or happen on a better quality solution (dominates)
- likelihood depends on problem space - breadth and height of peaks etc
- interaction may be subtle

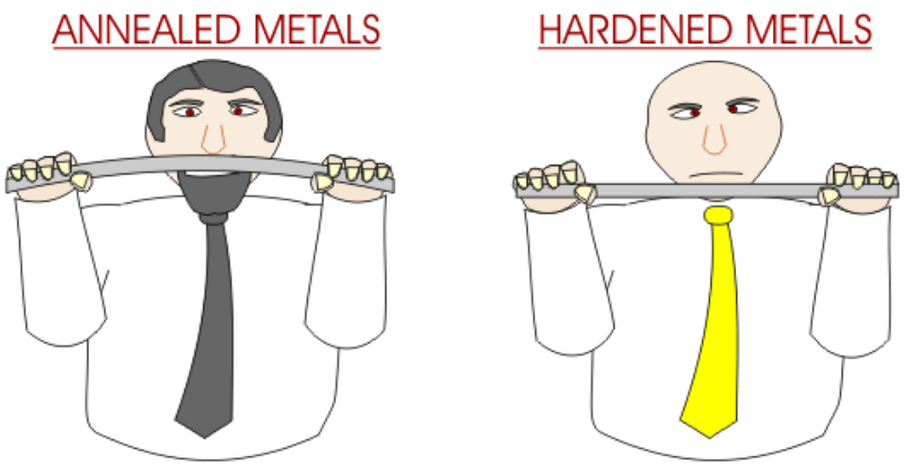
Algorithm 6 *Steepest Ascent Hill-Climbing With Replacement*

```
n ← number of tweaks desired to sample the gradient

S ← some initial candidate solution
Best ← S
repeat
    R ← Tweak(Copy(S))
    for n − 1 times do
        W ← Tweak(Copy(S))
        if Quality(W) > Quality(R) then
            R ← W
    S ← R
    if Quality(S) > Quality(Best) then
        Best ← S
until Best is the ideal solution or we have run out of time
return Best
```

		Noise in Tweak	
		Low	High
Samples	Few		Explorative
	Many	Exploitative	

Simulated Annealing





Simulated Annealing

- Developed for optimisation in the 1980s, though dates back to the 50s.
- Based on concept of annealing in metallurgy - atoms start more mobile and mobility decreases as the metal cools - rate of cooling affects outcome (in this case properties of the metal)
- Optimisation concept
  - Initially a lot of “energy” (heat) in the system - candidate(s) can jump around more
  - Energy reduces over time (cooling)
  - Smooth transition from exploration to exploitation

Simulated Annealing

- Similar to hill climbing, but can replace with current candidate S with a ***worse*** one R = Tweak(Copy(S)) according to probability

$$P(t, R, S) = e^{\frac{\text{Quality}(R) - \text{Quality}(S)}{t}} \qquad t \geq 0 \qquad (t \text{ is temp, not time!})$$

- Assume Quality(R) < Quality(S), then

$$P(t, R, S) = \frac{1}{e^{\frac{\text{Quality}(S) - \text{Quality}(R)}{t}}}$$

  - exponent is -ve
  - the worse R is, the smaller the probability of being accepted
  - the smaller t is, the smaller the probability of R being accepted
- t can be thought of as the "temperature"
- t is decreased over time, according to some *schedule*
- $t = \infty$: ***random walk***; $t = 0$: **hill climb**

➡ t is a very direct "tuning knob" between exploration and exploitation!

Simulated Annealing

Algorithm 13 *Simulated Annealing*

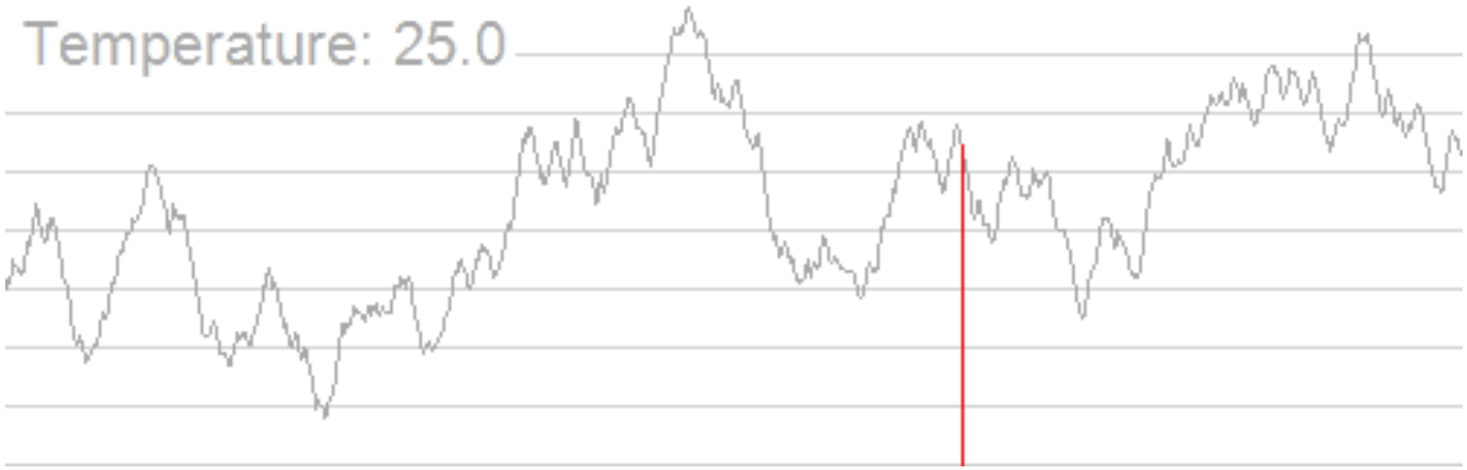
```
t ← temperature, initially a high number

S ← some initial candidate solution
Best ← S
repeat
    R ← Tweak(Copy(S))
    if Quality(R) > Quality(S) or if a random number chosen from 0 to 1 < e^((Quality(R)−Quality(S))/t) then
        S ← R
    Decrease t
    if Quality(S) > Quality(Best) then
        Best ← S
until Best is the ideal solution, we have run out of time, or t ≤ 0
return Best
```

- t is decreased according to some schedule - normally tuneable, eg

  $t = \beta e^{-\alpha T}$, where T is time

- hyper-parameter α determines rate of temperature decrease

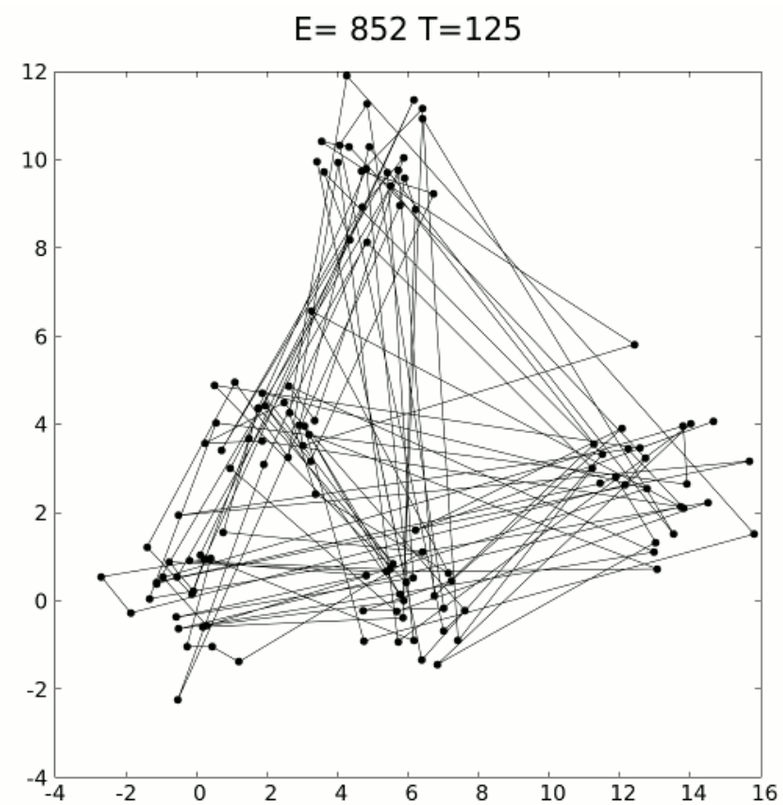
Simulated Annealing



By Kingpin13 - Own work, CC0, https://commons.wikimedia.org/w/index.php?curid=25010763

Can also be used on combinatorial problems.

eg. *Travelling salesman*



By Geodac - Own work, CC0, https://commons.wikimedia.org/w/index.php?curid=67988888

Tabu Search

- Glover, 1986
- Different approach to exploration
  - heuristic is basically “don’t go back where you’ve already been”
  - name comes from considering those places to be tabu (taboo)
  - *eventually* will get out of any local maximum (minimum)
  - clearly only works (in pure form) for **discrete spaces**, but there are workarounds
- ***tabu list*** - history of considered (*visited*) candidate solutions
  - linear in size (space)
  - have to search each iteration - linear in time
    - ➡ *trade-off* between ‘revisiting’ and checking the tabu list!

Tabu Search

- In practice, *tabu list L* - history of *recently* visited candidate solutions
  - maximum length *l*
    - ➡ constant time search (constant may still be large)
  - really a queue
    - oldest visited solutions “fall off” the end
- Following algorithm based on Steepest Ascent with Replacement (Alg 6)…

Tabu Search

Algorithm 14 *Tabu Search*

```
l ← Desired maximum tabu list length
n ← number of tweaks desired to sample the gradient

S ← some initial candidate solution
Best ← S
L ← {} a tabu list of maximum length l          ▷ Implemented as first in, first-out queue
Enqueue S into L
repeat
    if Length(L) > l then
        Remove oldest element from L
    R ← Tweak(Copy(S))
    for n − 1 times do
        W ← Tweak(Copy(S))
        if W ∉ L and (Quality(W) > Quality(R) or R ∈ L) then
            R ← W
    if R ∉ L then
        S ← R
        Enqueue R into L
    if Quality(S) > Quality(Best) then
        Best ← S
until Best is the ideal solution or we have run out of time
return Best
```

Tabu Search Variations

- Real-valued spaces
  - define “sufficiently close/similar”
  - reject if sufficiently close
  - may be expensive
  - may be infeasible for high-dimensional spaces
- Non-numerical feature spaces
  - create a tabu list of *changes recently made to features*
  - *Tweak* consults tabu list when making changes
  - can get complicated - eg. only the specific change that was made to a feature, or any changes to the feature
  - see Luke Alg. 15.

Iterated Local Search (ILS)

- Hill climbing with “cleverer”(?) random restarts
  - using a *heuristic* about where to restart
  - note that restarts are *attempts to find new local optima* (otherwise we’d just end up back at the top of the same hill - waste of compute)
  - ILS heuristic assumes “better local optima can be found near the one you’re in”
  - tries to “walk” from local optimum to local optimum
  - kind of a “meta-search”
  - *if* heuristic is “correct" (ie. appropriate for the space) it should out-perform random restarts (but remember NFL)

Iterated Local Search (ILS)

- "home base" - local optimum the vicinity of which we want to explore
  - select new restart locations that are "somewhat" in the vicinity of the home base
  - a good meaning of "somewhat" will again depend on the space
- When discovering a new optimum, decide whether to retain the current home base, or adopt the new optimum as home base
  - eg. only adopt new optimum if better than current - "hill climb of hill climbs"
  - eg. adopt every new optimum - "random walk of hill climbs"

Q. What does this decision remind you of?

Iterated Local Search (ILS)

Algorithm 16 *Iterated Local Search (ILS) with Random Restarts*

```
T ← distribution of possible time intervals

S ← some initial random candidate solution
H ← S                                          ▷ The current "home base" local optimum
Best ← S
repeat
    time ← random time in the near future, chosen from T
    repeat
        R ← Tweak(Copy(S))
        if Quality(R) > Quality(S) then
            S ← R
    until S is the ideal solution, or time is up, or we have run out of total time
    if Quality(S) > Quality(Best) then
        Best ← S
    H ← NewHomeBase(H, S)
    S ← Perturb(H)
until Best is the ideal solution or we have run out of total time
return Best
```

- Functions NewHomeBase() and Perturb() make the decisions that determine how the heuristic works...

Iterated Local Search (ILS)

Designing NewHomeBase() and Perturb() is, as Luke states, a "black art" (in fact a lot of choosing meta-heuristics/hyper-parameters is!)

- NewHomeBase()
  - one extreme, always adopt new home base - "random walk"

$$\text{NewHomeBase}(H, S) = S$$

  - another extreme, only adopt if at least as good - "hill climb"

$$\text{NewHomeBase}(H, S) = \begin{cases} S & \text{if Quality}(S) \geq \text{Quality}(H) \\ H & \text{otherwise} \end{cases}$$

  - in practice might go for a "happy medium"
  - or perhaps use a simulated annealing approach!

Iterated Local Search (ILS)

- Perturb()
  - needs to be “big enough” but “not too big”
  - depends strongly on the problem

Q: Might a simulated annealing approach work here too?

Mixing and Matching

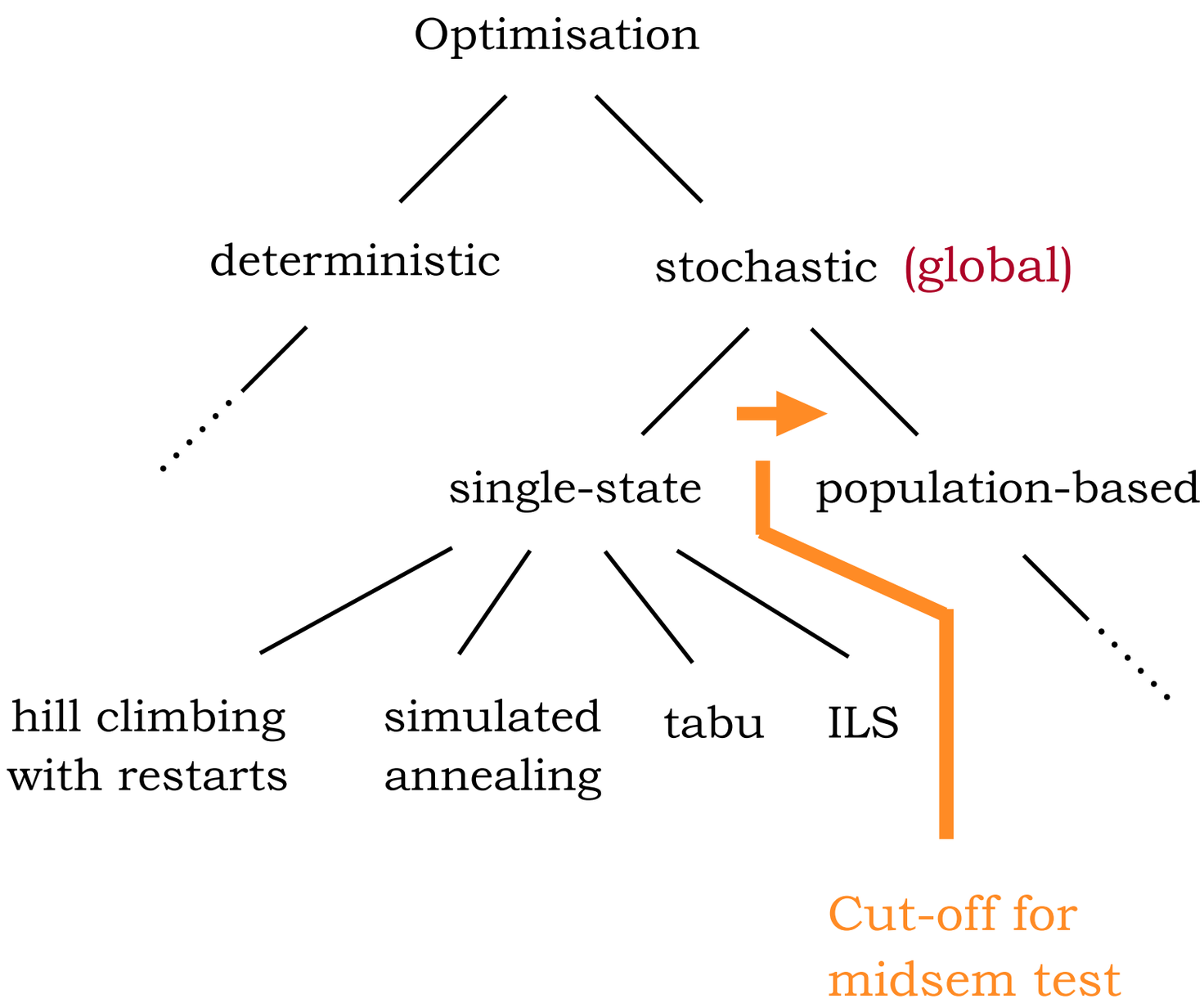
- This example shows how the algorithms can be mixed and matched to solve a problem.
- You can no doubt think of other things you could try - only limit is your imagination!

Some Key Things to Think About

- How do you know if your algorithm is any good?
- How do you compare algorithms? Hyper-parameters? What metrics?
  - Computing resource?
  - Evaluations?
  - Quality of solution?
  - Over what *class* of problems? (*cf.* NFL)
- Is optimisation a theoretical or an empirical (experimental) science?

Some Key Things to Think About

- Are we throwing away information (and wasting compute) in single-state methods?
- How much "remembering" or "knowledge" of search space information is encoded in our (chosen) single-state method?
  - *implicit* or *explicit* ?
    - Examples?
- Could we make better use of it to learn about the problem space??

Next - population-based methods!

Optimisation Algorithms - Roadmap